Trip Genius AI — Project Documentation

AI/ML and Generative AI Track • Project Documentation

Project	Trip Genius AI (VoyagePlan Travel Planner)	**Date**	26 October 2026
Submitted by	Kumkum Thakur (individual project)	**Hall Ticket No.**	0157CS241131
College	LNCTS	**University**	RGPV

Project owner

Field	Details
Name	Kumkum Thakur
Hall ticket number	0157CS241131
College	LNCTS
University	RGPV
Role	Sole developer: design, frontend, backend, AI integration, database, deployment and testing

Abstract

Trip Genius AI is a web-based travel itinerary application. A traveller signs in, submits destination, dates, interests, budget and pace, and receives an AI-generated day-wise itinerary produced by a FastAPI backend that calls Groq. The traveller can save the trip, revisit it in My Trips and edit its preferences. Supabase provides authentication and persistent storage. The frontend (React + Vite) is hosted on Vercel and the backend on Render.

1. Introduction

Planning a trip involves choosing activities, arranging them over available days, balancing interests with budget and keeping the plan updated as preferences change. Trip Genius AI streamlines this by collecting requirements in one place and returning a structured itinerary that can be saved and edited.

2. Problem statement

Travel information is fragmented across search results, blogs, maps and booking services, and travellers must manually synthesise it into a day-by-day schedule. Notes are hard to revisit or update. The project provides a centralised planning workflow, but does not replace independent verification of opening hours, availability, prices or travel conditions.

3. Objectives

- Provide a clear travel-preference form with validation.
- Generate a destination-relevant multi-day itinerary through a backend API using Groq.
- Support account creation and sign-in using Supabase Auth.
- Allow users to save, list, reopen and edit their trips with persistence across refreshes.
- Deploy frontend and backend as separate hosted services and keep provider credentials server-side.

4. Scope

Included: web UI, preference collection, API request / response flow, AI generation, authentication, saved trips, edit flow and hosted deployment. **Not included:** booking or payment, guaranteed real-time pricing, live flight / hotel inventory, map route optimisation.

5. Users and use cases

Actor	Use case	Outcome
Traveller	Register / sign in	Authenticated session
Traveller	Enter trip preferences	Validated generation request
Traveller	Generate itinerary	Day-wise itinerary returned by backend
Traveller	Save trip	Trip persisted for later
Traveller	View My Trips	Saved journeys listed
Traveller	Edit preferences	Saved values restored for editing
Evaluator	Open live app and repository	Understands and verifies the implementation

6. System architecture

Frontend: React + Vite on Vercel. Backend: Python + FastAPI on Render. Authentication and database: Supabase Auth and PostgreSQL. AI integration: Groq API called only from the backend. Communication uses HTTPS and CORS allows the exact production frontend origin.

Layer	Technology	Responsibility
Presentation	React + Vite	Pages, forms, navigation, loading / error states, itinerary rendering
API / business logic	FastAPI + Pydantic	Validation, orchestration, protected operations, response shaping
AI generation	Groq API	Generates itinerary content from preferences
Identity	Supabase Auth	Sign-up, sign-in, session / token lifecycle
Data	Supabase PostgreSQL	Saved trips
Hosting	Vercel + Render	Frontend and backend deployment

7. Technology stack

Technology	Purpose
React + Vite	Client-side application and build tooling
FastAPI + Python 3.11	REST API and generation workflow
Pydantic	Request / response validation
Groq API	LLM inference for itinerary generation
Supabase	Authentication and persistent trip data
Vercel / Render	Frontend / backend hosting
Git + GitHub	Source control

8. Core workflow

1. User signs up or signs in.
2. User enters destination, dates, interests, budget and pace.
3. Frontend sends the request to POST /api/itineraries/generate.
4. Backend validates the request with Pydantic and calls Groq.
5. Frontend renders the returned day-wise itinerary.
6. User saves the trip; it is stored for the authenticated user in Supabase.
7. User opens My Trips, selects a trip and edits its preferences; data persists after refresh.

9. API overview

Endpoint	Purpose
GET /health	Backend health check (returned healthy).
POST /api/itineraries/generate	Validate trip preferences and return the generated itinerary.
Saved-trip operations	Save, list, read and update trips for the authenticated user via the Supabase-backed data layer.

10. Data and security

- Supabase Auth manages credentials and sessions; no custom password storage.
- Trips are tied to the authenticated user; Row Level Security and backend ownership checks are used to restrict access, and a two-account cross-access test is a planned verification step.
- Groq key and any service-role key stay on Render; the browser only holds the public anon key.
- CORS allows only the deployed Vercel origin; .env files are never committed.

11. Deployment

Field	Details
Frontend	https://travel-itinerary-planner-0023-b9jioymcx-kumkum18.vercel.app
Backend	https://travel-itinerary-planner-1ysc.onrender.com
Health endpoint	https://travel-itinerary-planner-1ysc.onrender.com/health
Supabase	https://wwhitxfzfdvqwgogqsuj.supabase.co
Repository	https://github.com/thakurkumkum/travel-itinerary-planner
Render settings	Root: backend | Build: pip install -r requirements.txt | Start: uvicorn app.main:app --host 0.0.0.0 --port $PORT | Python 3.11.9

12. Testing

Production smoke tests were run on the live deployment and all passed: sign-up / sign-in, itinerary generation, saving a trip, listing it in My Trips, restoring preferences for editing, persistence after refresh and a clean browser console. The health endpoint also returned healthy. These are manual functional checks; quantitative load-test results have not been collected yet.

13. Limitations

- AI-generated suggestions may be inaccurate or outdated and must be verified by the traveller.
- AI latency, quality and cost depend on Groq availability and rate limits; the Render service may cold-start after inactivity depending on its plan.
- Live prices, opening hours and booking availability are not provided.
- No automated test suite or measured performance figures yet.

14. Future enhancements

Priority	Enhancement
High	Cross-account ownership / RLS verification, AI timeout and retry handling, structured itinerary schema, usage limits
Medium	Map-based activity grouping, budget breakdown, PDF export, shareable trip links, finer preference controls
Later	Live data integrations (weather, hours, prices), collaborative planning, analytics and observability

15. Conclusion

Trip Genius AI delivers a deployed, end-to-end workflow for planning, saving and editing travel itineraries using React, FastAPI, Groq and Supabase. Production smoke tests support the core user journey. Further security review, automated testing and performance measurement would strengthen readiness for wider use.

Appendix: project links

Field	Details
Live application	https://travel-itinerary-planner-0023-b9jioymcx-kumkum18.vercel.app
GitHub repository	https://github.com/thakurkumkum/travel-itinerary-planner
Backend health	https://travel-itinerary-planner-1ysc.onrender.com/health
Demo video	Not recorded yet